

One canteen bill. Everything in Module 1.

How a computer turns Riya's order into a printed bill, and every concept it uses on the way.

TODAY'S CASE

Riya's canteen bill

Riya orders 2 chai, 1 samosa and 1 Maggi. In under a second, the counter machine prints her bill.

Every topic of Module 1 is hiding inside that one second.

IIIT CANTEEN

Bill No. 0075

Chai × 2	₹20
Samosa × 1	₹15
Maggi × 1	₹40
TOTAL	₹75

Thank you!

THE ROUTE

Five stops through Module 1



One bill. Five stops. Every topic of the module.

01

The machine

Who calculated the canteen bill before computers existed?

STOP 1 • THE MACHINE

From beads to billions

Ancient	1642	1837	1843	1946	1971	1991	Today
Abacus: beads slide to add and subtract	Pascal's calculator: gears add numbers	Babbage designs the Analytical Engine, a programmable machine	Ada Lovelace writes the first published algorithm	ENIAC: about 18,000 vacuum tubes, filling a room	Intel 4004: a whole CPU on one chip	PARAM 8000: India's first supercomputer	A phone in your pocket

The job never changed: **input, calculate, remember, output**. Only the machine got smaller and faster.

STOP 1 • THE MACHINE

Five generations of computers

1st

Vacuum tubes

1940s–50s

Room-sized, hot,
programmed with
switches and
punched cards

2nd

Transistors

Late 1950s–60s

Smaller, cooler,
more reliable

3rd

Integrated circuits

1960s–70s

Many transistors on
one chip; keyboards
and monitors

4th

Microprocessors

1970s–now

A whole CPU on
one chip: PCs,
laptops, phones

5th

AI and beyond

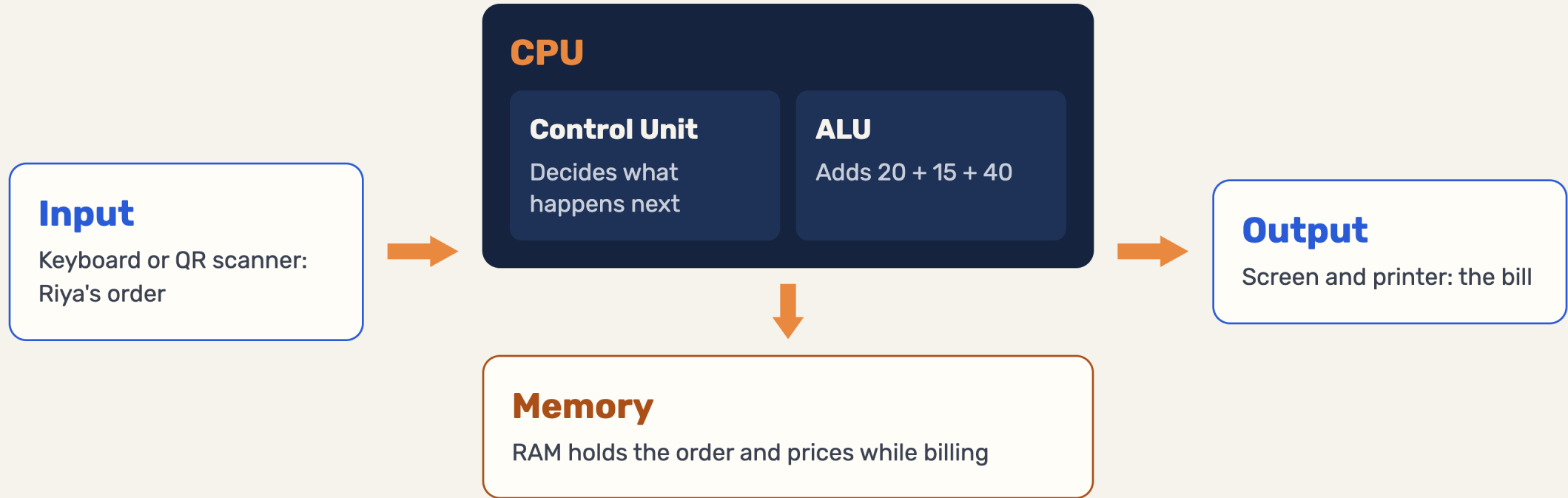
Now

Machines that learn
patterns from data

Each generation: **smaller, faster, cheaper** and easier to program.

STOP 1 • THE MACHINE

Inside the billing machine



STOP 1 • THE MACHINE

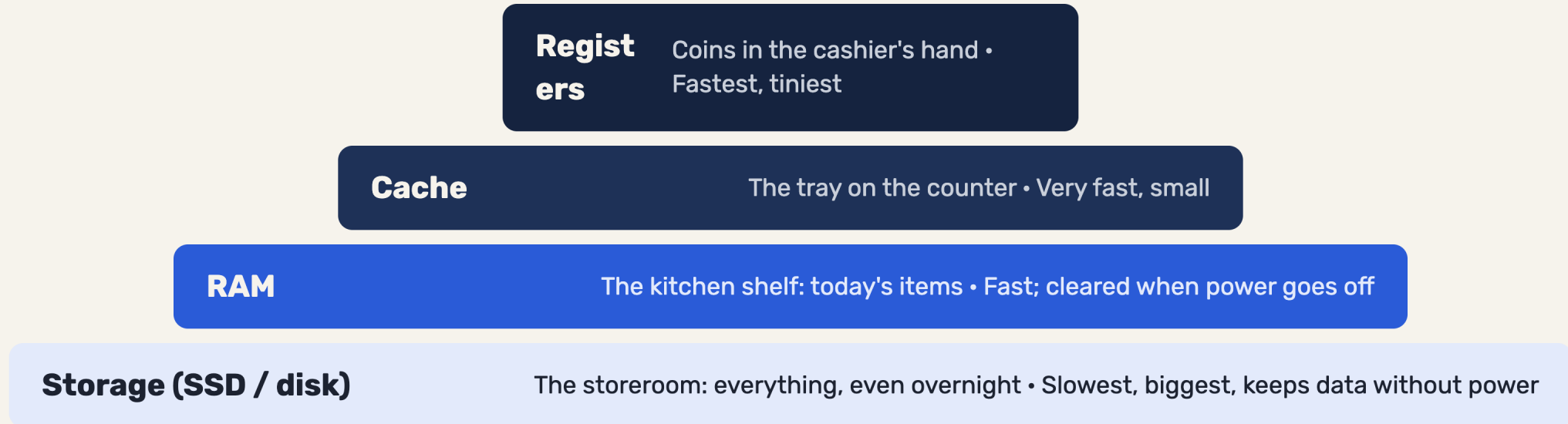
Watch the bill move through the machine



1 • INPUT: Riya's order is typed at the counter

STOP 1 • THE MACHINE

Memory: fast and small, or slow and big



The CPU keeps what it needs **right now** close by, and everything else further away.

STOP 1 • THE MACHINE • QUICK CHECK

Can you answer these?

1 Which part of the computer calculates $20 + 15 + 40$?

The ALU, inside the CPU

2 Where is the price list kept when the machine is switched off?

Storage (SSD or hard disk)

3 Is the bill printer an input or an output device?

Output

02

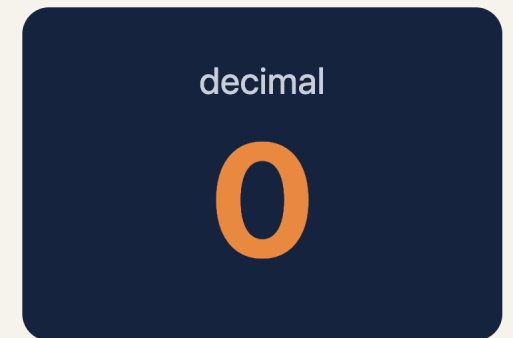
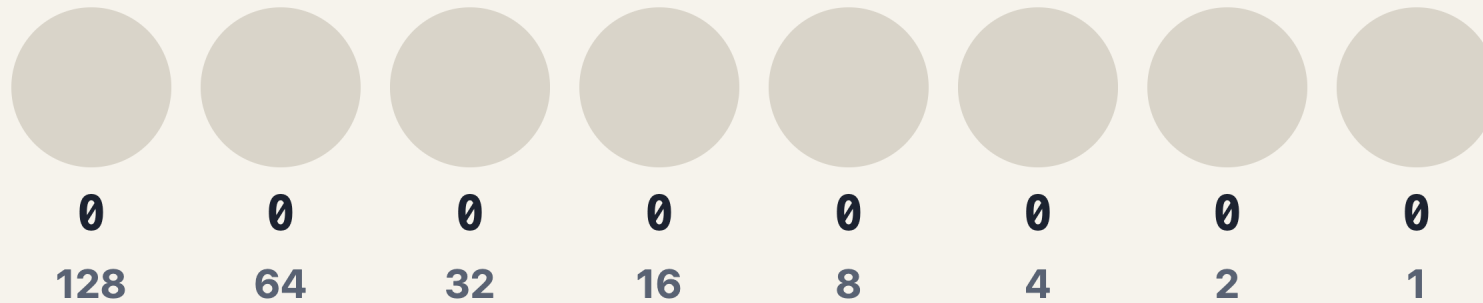
The data

The machine only knows ON and OFF. So how does it store ₹75, the word CHAI, or a photo?

STOP 2 • THE DATA

Everything is ON or OFF: bits

1 bit = one switch. 8 bits = 1 byte = 256 different patterns, from 0 to 255.



0 0 0 0 0 0 0 0

Every bulb OFF: 0

STOP 2 • THE DATA

Same ₹75, two number systems

**Decimal
base 10**

x10	x1
7	5

$$7 \times 10 + 5 \times 1 = \mathbf{75}$$

**Binary
base 2**

128	64	32	16	8	4	2	1
0	1	0	0	1	0	1	1

$$64 + 8 + 2 + 1 = \mathbf{75}$$

STOP 2 • THE DATA

Decimal to binary: keep dividing by 2

75 ÷ 2 = 37 remainder **1**

37 ÷ 2 = 18 remainder **1**

18 ÷ 2 = 9 remainder **0**

9 ÷ 2 = 4 remainder **1**

4 ÷ 2 = 2 remainder **0**

2 ÷ 2 = 1 remainder **0**

1 ÷ 2 = 0 remainder **1**



Read the remainders from bottom to top

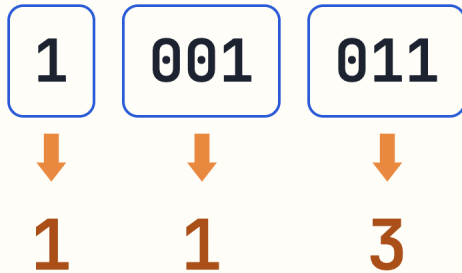
1001011

75 in decimal = 1001011 in binary

STOP 2 • THE DATA

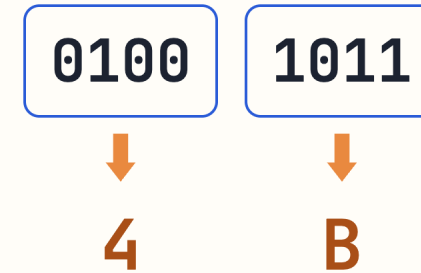
Octal and hex: binary in short form

Octal: groups of 3 bits



75 = **113** in base 8

Hexadecimal: groups of 4 bits



75 = **4B** in base 16

Hex digits after 9: **A=10, B=11, C=12, D=13, E=14, F=15**. So 1011 = 11 = B.

STOP 2 • THE DATA

Where you'll meet octal and hex



Colours

#E9893F

E9 = 233 red, 89 = 137 green, 3F = 63 blue. The orange in these slides.

```
$ chmod 755 bill
```

File permissions

755 (octal)

7 = 111 = read, write, run. Each octal digit is 3 bits.

```
&total = 0x7ffd4c
```

Memory addresses

0x... (hex)

Debuggers show addresses in hex. Pointers in Module 5 use them.

STOP 2 • THE DATA

Text is numbers too: CHAI in ASCII

C



67

01000011

H



72

01001000

A



65

01000001

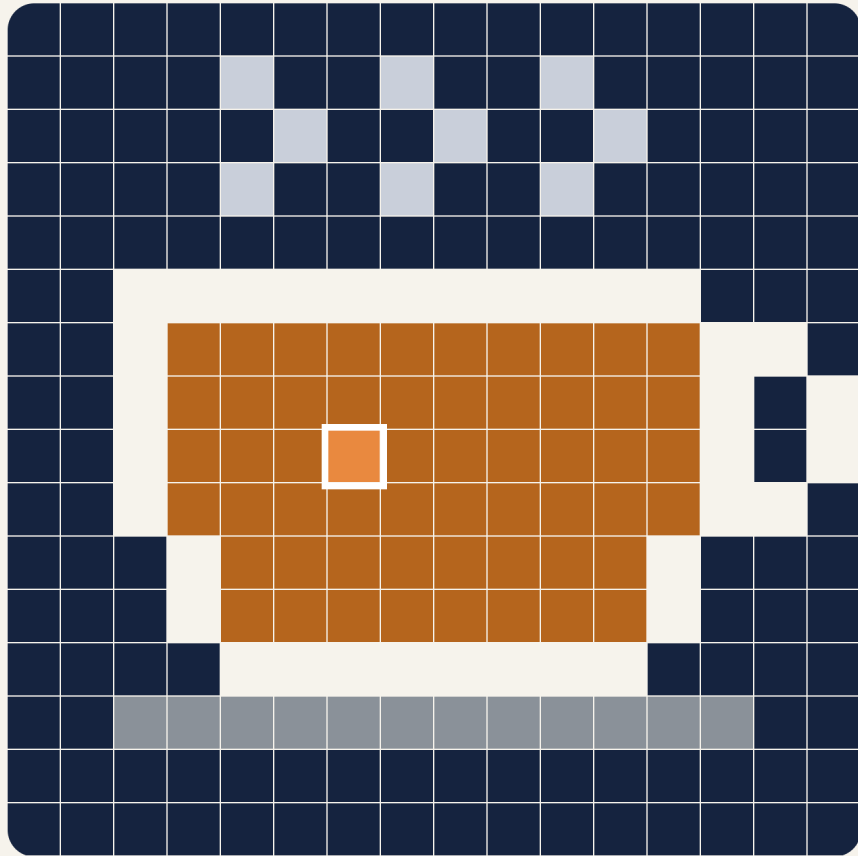
I



73

01001001

ASCII gives every English character a number: A = 65, B = 66 ... **Unicode** extends this to हिंदी, ಕನ್ನಡ and emoji.



STOP 2 • THE DATA

Pictures are numbers too

An image is a grid of pixels. Each pixel is three numbers: **red**, **green**, **blue**, each 0 to 255.

Red

233

Green

137

Blue

63

A 1920 × 1080 photo has about 2 million pixels, so about 6 million bytes before compression.

STOP 2 • THE DATA • QUICK CHECK

Can you convert these?

1 1010 in binary = ? in decimal

10

2 How many different values can 8 bits store?

256 (0 to 255)

3 What is 255 in hexadecimal?

FF

03

The language

The CPU understands only bits. How do we tell it to add up Riya's bill?

STOP 3 • THE LANGUAGE

One instruction, four levels

You (English)

Add the price of chai and samosa.

High-level language (C)

```
total = chai + samosa;
```

Assembly language

```
MOV AX, chai  
ADD AX, samosa  
MOV total, AX
```

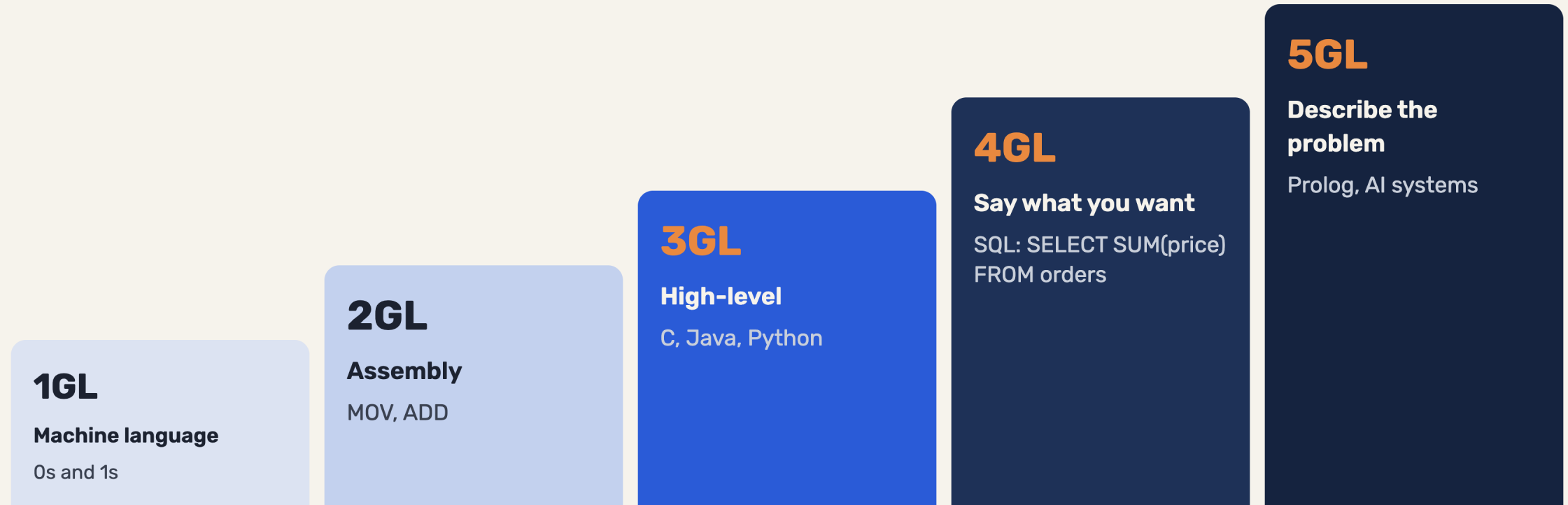
Machine language

```
10100001 00001010  
00000011 00001100
```

Assembly and machine code shown are illustrative; real codes depend on the processor.

STOP 3 • THE LANGUAGE

Generations of programming languages



Each generation moves **closer to how humans think** and further from the hardware.

STOP 3 • THE LANGUAGE

Translators: compiler and interpreter



Compiler

Like translating the **whole menu book** into Hindi, then printing it. Slow to start; afterwards anyone reads it instantly.

C, C++



Interpreter

Like a **live interpreter** for a visitor: translates one sentence, it is spoken, then the next.

Python, BASIC

STOP 3 • THE LANGUAGE

Watch them work: compiler vs interpreter

COMPILER

```
1 chai = 2 * 10
2 samosa = 15
3 total = chai + samosa
4 print total
5 print "Thank you"
```



INTERPRETER

```
1 chai = 2 * 10
2 samosa = 15
3 total = chai + samosa
4 print total
5 print "Thank you"
```



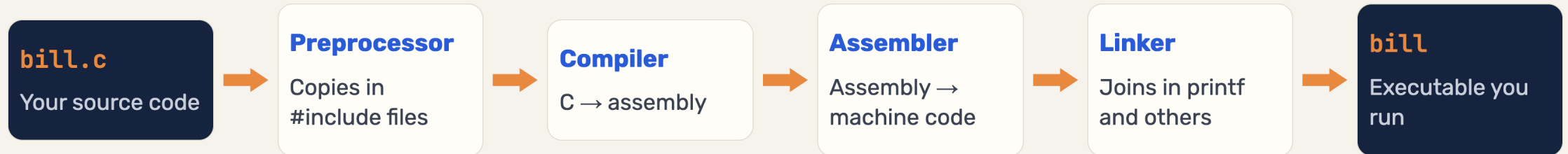
STOP 3 • THE LANGUAGE

Compiler vs interpreter at a glance

	Compiler	Interpreter
Translates	Whole program at once	One line at a time
Errors reported	All, before running	At the faulty line, while running
Creates a file to run?	Yes (bill.exe or a.out)	No
Running speed	Fast	Slower
Examples	C, C++	Python, BASIC

STOP 3 • THE LANGUAGE

From bill.c to a program you can run



```
$ gcc bill.c -o bill
$ ./bill
TOTAL ₹75
```

One gcc command runs all four steps. In Module 2 you will watch each one separately.

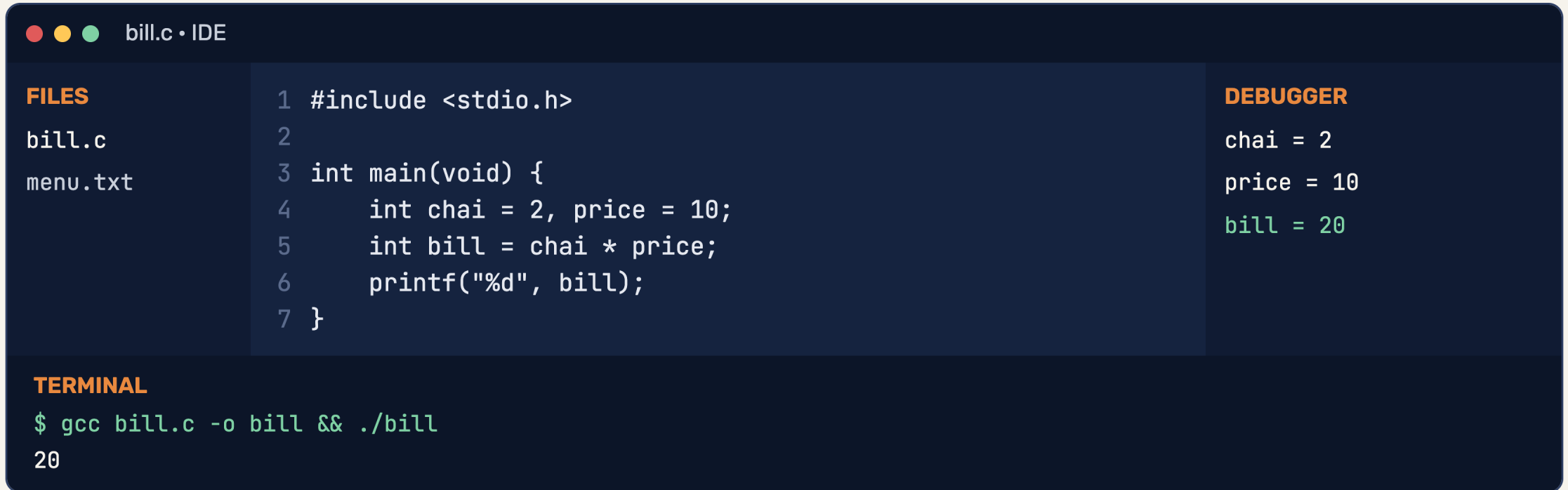
04

The mistakes

Riya's bill prints ₹12 instead of ₹20. What went wrong, and how do we find it?

STOP 4 • THE MISTAKES

The programming environment



Editor: write code

Compiler + terminal: build and run

Debugger: watch variables

STOP 4 • THE MISTAKES

Four kinds of errors in the bill program

Syntax error

Found by: the compiler

```
total = chai + samosa
```

error: expected ';'

Linker error

Found by: the linker

```
int mian(void) { ... }
```

undefined reference to 'main'

Runtime error

Found: while running

```
share = total / friends;
```

friends = 0 → program crashes

Logical error

Found by: only you

```
bill = chai + price;
```

2 chai at ₹10 → ₹12 ×

STOP 4 • THE MISTAKES • NAME THE ERROR

Which type of error is it?

1 gcc stops and says: expected ';' before 'return'

Syntax error

2 The bill for 2 chai at ₹10 prints ₹12

Logical error

3 It compiles, then: undefined reference to 'main'

Linker error

4 Splitting the bill among 0 friends crashes the program

Runtime error

STOP 4 • THE MISTAKES

Four ways to hunt a bug



Print debugging

Print values to see what the program is really doing.

```
printf("%d", bill);
```



Dry run

Trace the program by hand with a table, line by line.

```
chai | price | bill
```



Debugger

Pause at a breakpoint, step line by line, watch variables.

```
break, step, watch
```



Rubber duck

Explain your code aloud, line by line. The bug often jumps out.

```
"First I add..."
```

STOP 4 • THE MISTAKES

A debugger, step by step

```
1 int prices[] = {10, 10, 15, 40};  
2 int total = 0;  
3 for (int i = 0; i < 4; i++) {  
4     total = total + prices[i];  
5 }  
6 printf("Total: %d", total);
```

VARIABLES

i	—
prices[i]	—
total	—

Start: prices stored in an array

05

The thinking

Before any code: how do we plan the bill program so that it is right the first time?

STOP 5 • THE THINKING

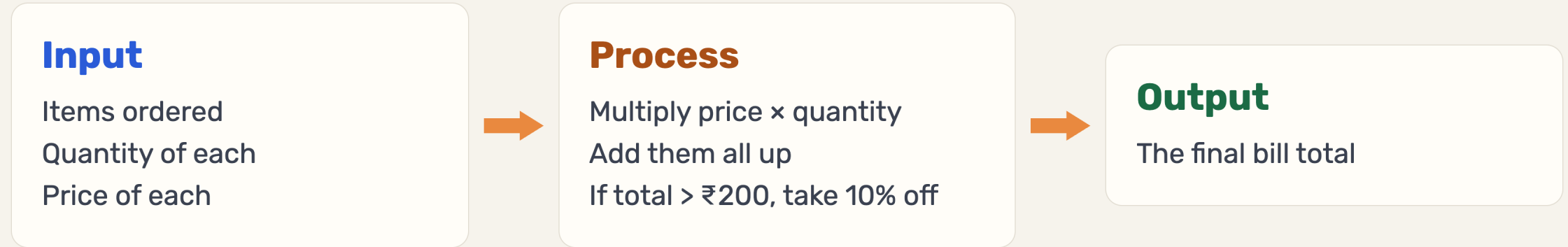
Six steps of problem solving



Steps 1 to 3 happen **before you touch the keyboard**. Remember: 99% thinking, 1% typing.

STOP 5 • THE THINKING

Analyse the bill: input, process, output



Test case	Before discount	Expected bill
Riya's order	₹75	₹75
Hostel group order	₹250	₹225
Exactly ₹200	₹200	₹200 (no discount)

STOP 5 • THE THINKING

The bill as an algorithm

- 1 Start.
- 2 Set total to 0.
- 3 For each item: read its price and quantity, and add price × quantity to total.
- 4 If total is more than 200, take 10% off total.
- 5 Print total.
- 6 Stop.

A good algorithm is...

- | | |
|------------------|---------------------------------|
| Input | zero or more values go in |
| Output | at least one result comes out |
| Definite | every step is clear |
| Finite | it always stops |
| Effective | every step can actually be done |

STOP 5 • THE THINKING

Flowchart symbols



Terminal

Start or stop

Start



Input / Output

Read or print

Read price



Process

A calculation

total = total + 20



Decision

A yes/no question

total > 200?



Flow line

Shows the order

next step



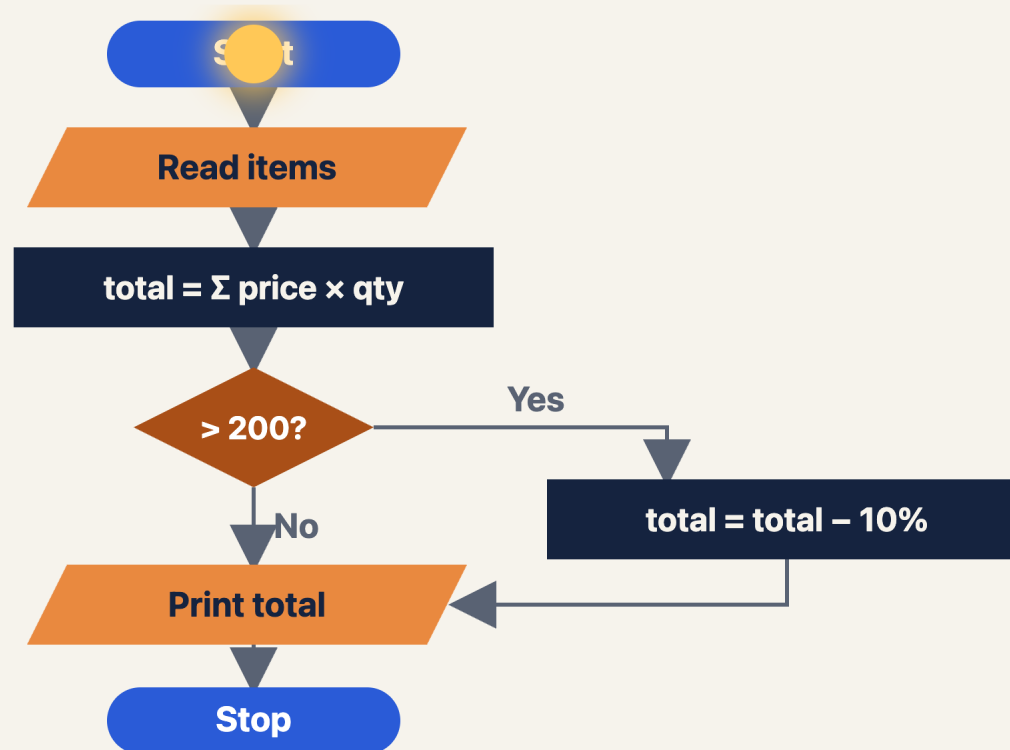
Connector

Joins parts of a
long chart

A

STOP 5 • THE THINKING

The bill flowchart, running



Case 1 • Riya's order

total = —

Start



STOP 5 • THE THINKING

Algorithm → pseudocode → C

Algorithm (plain words)	Pseudocode	C (coming in Module 2)
Set total to 0	<code>total ← 0</code>	<code>int total = 0;</code>
For each item, add price × quantity	<code>FOR each item DO total ← total + price × qty END FOR</code>	<code>for (i = 0; i < n; i++) total += price[i] * qty[i];</code>
If total is more than 200, take 10% off	<code>IF total > 200 THEN total ← total – total / 10 END IF</code>	<code>if (total > 200) total = total - total / 10;</code>
Print total	<code>PRINT total</code>	<code>printf("%d", total);</code>

Pseudocode is the bridge: **structured like code, but no strict syntax.**

MODULE 1 IN ONE PICTURE

Everything we saw inside one bill

